# TypeScript Basic Problem-Solving

## General Rules

- **Do not change the function names.** Use the exact name given in each problem.
- Every problem must be solved using a **function**.
- **Every function must return a value.** Do not use `console.log()` as the final answer.
- Use appropriate TypeScript types for function parameters, function return values, objects, arrays, and union types where appropriate.
- Avoid using `any` unless a problem explicitly requires it.
- You may use JavaScript/ES6 features such as `map()`, `filter()`, `reduce()`, `find()`, `includes()`, destructuring, and template literals.
- Test your functions with the provided examples and create a few additional test cases yourself.
- Think about edge cases before considering a problem completely.

## Problem 1 — Cinema Ticket Counter

*Concepts: function parameter types, return types, number, conditional logic*

**Function name must be: `getTicketPrice`**

### Scenario

A cinema is building a ticketing system. The ticket price depends on the customer's age because children and senior citizens receive special pricing. You are responsible for creating the function that determines how much a customer should pay for a single ticket.

### Task

Create a function named `getTicketPrice`. The function should receive the customer's age and return the appropriate ticket price.

### Pricing Rules

| Age | Ticket Price |
|---|---|
| Below 5 | 0 |
| 5–12 | 100 |
| 13–59 | 200 |
| 60 or above | 120 |

A child younger than 5 years old can enter for free.

### Function Requirement

- Accept the age as a `number`.
- Return the ticket price as a `number`.
- Correctly handle all age ranges.

**Starter Code**

```
function getTicketPrice(age:<type>): <type> {

    // write your code here

}
```

**Example**

```
getTicketPrice(3);
// 0

getTicketPrice(10);
// 100

getTicketPrice(25);
// 200

getTicketPrice(65);
// 120
```

## Problem 2 — Store Inventory Status

*Concepts: function types, number, string, conditional logic, boundary conditions*

**Function name must be: `getStockStatus`**

### Scenario

An online store wants to show customers whether a product is available before they place an order. The inventory system stores the number of currently available units. Your job is to convert that number into a meaningful status message.

### Task

Create a function named `getStockStatus`. The function receives the current stock quantity and returns a status string.

### Status Rules

| Stock | Status |
|---|---|
| 0 | "Out of Stock" |
| 1–5 | "Almost Sold Out" |
| 6–20 | "Available" |
| More than 20 | "In Stock" |

### Additional Requirement

The function should return a `string`.

Think carefully about boundary values such as:

**Starter Code**

```
function getStockStatus(stock:<type>): <type>{

    // write your code here

}
```

**Example**

```
getStockStatus(0);
// "Out of Stock"

getStockStatus(3);
// "Almost Sold Out"

getStockStatus(12);
```

```
// "Available"

getStockStatus(50);
// "In Stock"

// boundary values to double-check:
getStockStatus(5);
getStockStatus(6);
getStockStatus(20);
getStockStatus(21);
```

## Problem 3 — Social Media Profile Formatter

*Concepts: object types, type/interface, function parameter typing, return types, template literals*

**Function name must be: `formatUserProfile`**

### Scenario

A social media application stores basic information about its users. The application needs a function that converts a user's structured data into a human-readable profile summary. A user contains `name`, `age`, and `city`.

### Task

First define an appropriate TypeScript type or interface for the user. Then create a function named `formatUserProfile`. The function should receive a user object and return a formatted sentence.

### Requirements

- Accept a properly typed user object.
- Return a `string`.
- Use the values from the object rather than hard-coding the result.

**Starter Code**

```
// TODO: define a type or interface for the user

function formatUserProfile(user: User): <type> {

    // write your code here

}
```

**Example**

```
formatUserProfile({
    name: "Fahim",
    age: 22,
    city: "Dhaka"
});

// Expected output:
// "Fahim is 22 years old and lives in Dhaka."
```

## Problem 4 — Shopping Cart Calculator

*Concepts: object types, array types, reduce(), function parameter and return types*

**Function name must be: `calculateCartTotal`**

### Scenario

An online shopping website stores each item in a customer's cart as an object containing its name and price. The store needs a function that calculates the total price of all products currently in the cart.

### Product Structure

```
{
    name: string;
    price: number;
}
```

### Task

Create a function named `calculateCartTotal`. The function should receive an array of products and return the total price.

### Requirements

- Properly type the product object.
- Properly type the array of products.
- Return the total as a `number`.
- An empty cart should return `0`.

**Starter Code**

```
// TODO: define a type for a single product

function calculateCartTotal(products: Product[]): <type> {

    // write your code here

}
```

**Example**

```
const products = [
    { name: "Keyboard", price: 1500 },
    { name: "Mouse", price: 800 },
    { name: "USB Cable", price: 300 }
];

calculateCartTotal(products);
// 2600
```

```
// another example:
const products2 = [
    { name: "Book", price: 500 },
    { name: "Pen", price: 50 },
    { name: "Bag", price: 1200 }
];

calculateCartTotal(products2);
// 1750
```

## Problem 5 — Student Result Analyzer

*Concepts: nested arrays, object types, reduce(), return object types, conditional logic*

**Function name must be: `getStudentResult`**

### Scenario

A school stores the marks of each student in an array. Teachers want a quick summary containing the student's name, average mark, and whether the student passed or failed. A student is considered **passed** if their average mark is at least 40.

### Student Structure

```
{
    name: string;
    marks: number[];
}
```

### Task

Create a function named `getStudentResult`. The function should receive a student object, calculate the average of all marks, determine whether the student passed, and return a new object containing `name`, `average`, and `result`.

### Edge Case

Think about what your function should do if the marks array is empty.

**Starter Code**

```
// TODO: define a type for a student

function getStudentResult(student: Student): <type> {

    // write your code here

}
```

**Example**

```
getStudentResult({
    name: "Rafi",
    marks: [80, 75, 90, 85]
});

// Expected output:
// { name: "Rafi", average: 82.5, result: "Passed" }

// another example:
getStudentResult({
    name: "Nabil",
    marks: [30, 35, 40, 25]
```

```
});

// Expected output:
// { name: "Nabil", average: 32.5, result: "Failed" }
```

## Problem 6 — Role-Based Permission Checker

*Concepts: union types, literal types, function parameter types, type safety*

**Function name must be: `canEdit`**

### Scenario

A web application has three types of users: `admin`, `editor`, and `viewer`. Different roles have different permissions. For this problem, only administrators and editors are allowed to edit content.

### Task

First create a union type `Role`, then create a function named `canEdit` that receives a valid `Role` and returns whether that role can edit content.

### Rules

| Role | Can Edit? |
|---|---|
| admin | true |
| editor | true |
| viewer | false |

### TypeScript Requirement

This should produce a TypeScript error — `canEdit("guest")` — since the purpose is to make TypeScript restrict the function to known roles.

**Starter Code**

```
function canEdit(role: Role): <type> {

    // write your code here

}
```

**Example**

```
canEdit("admin");
// true

canEdit("editor");
// true

canEdit("viewer");
// false
```

## Problem 7 — Product Category Search

*Concepts: typed arrays, object types, function parameters, filter(), return types*

**Function name must be: `findProducts`**

### Scenario

An e-commerce platform contains thousands of products. A customer selects a category, and the application needs to show only the products belonging to that category. Each product has a `name`, `price`, and `category`.

### Task

Create a function named `findProducts`. The function should receive an array of products and a category, and return all products that belong to that category.

### Requirement

If no product matches the category, return an empty array.

**Starter Code**

```
// TODO: define a type for a product (including category)

function findProducts(products: Product[], category: <type>): <type> {

    // write your code here

}
```

**Example**

```
const products = [
    { name: "iPhone 15", price: 90000, category: "phone" },
    { name: "Galaxy S24", price: 85000, category: "phone" },
    { name: "MacBook Air", price: 120000, category: "laptop" },
    { name: "Dell XPS", price: 110000, category: "laptop" }
];

findProducts(products, "phone");
// returns the iPhone 15 and Galaxy S24 objects

findProducts(products, "laptop");
// returns the two laptop products
```

## Problem 8 — Hospital Patient Status

*Concepts: union types, optional properties, type narrowing, discriminated unions, object types*

**Function name must be: `getPatientStatus`**

### Scenario

A hospital has two types of patients: general patients and emergency patients. General patients only have basic information. Emergency patients have an additional `emergencyLevel`.

### General Patient

```
{
    name: "Rahim",
    age: 35,
    type: "general"
}
```

### Emergency Patient

```
{
    name: "Karim",
    age: 60,
    type: "emergency",
    emergencyLevel: 1
}
```

Emergency levels are: `1` → Critical, `2` → Serious, `3` → Moderate.

### Task

Create a function named `getPatientStatus`. The function should receive either a general patient or an emergency patient and return an appropriate status message.

### TypeScript Requirement

Use TypeScript's type system to represent the fact that `emergencyLevel` exists for emergency patients but not necessarily for general patients.

**Starter Code**

```
// TODO: define types for GeneralPatient and EmergencyPatient

function getPatientStatus(patient: GeneralPatient | EmergencyPatient):
<type> {

```

```
    // write your code here

}
```

**Example**

```
getPatientStatus({ name: "Rahim", age: 35, type: "general" });
// "General patient"

getPatientStatus({ name: "Karim", age: 60, type: "emergency",
emergencyLevel: 1 });
// "Critical emergency"

getPatientStatus({ name: "Hasan", age: 45, type: "emergency",
emergencyLevel: 3 });
// "Moderate emergency"
```

## Problem 9 — Bank Transaction Processor

*Concepts: union types, discriminated unions, type narrowing, object types, return types, conditional logic*

**Function name must be: `processTransaction`**

### Scenario

A banking application needs to process deposits and withdrawals. Every transaction contains a `type` and an `amount`.

### Deposit

```
{
    type: "deposit",
    amount: 2000
}
```

### Withdrawal

```
{
    type: "withdraw",
    amount: 1500
}
```

### Task

Create a function named `processTransaction`. It should receive the current account balance and a transaction, and return the new balance.

### Rules

- A **deposit** increases the balance.
- A **withdrawal** decreases the balance.
- A customer **cannot withdraw more money than they currently have** — in that case, the original balance should remain unchanged.

### TypeScript Requirement

Represent the two possible transaction shapes using TypeScript's type system. The function should not accept arbitrary transaction types.

**Starter Code**

```
// TODO: define types for Deposit and Withdrawal transactions

function processTransaction(balance: <type>, transaction: Deposit |
Withdrawal): <type> {

    // write your code here
```

```
}
```

**Example**

```
processTransaction(5000, { type: "deposit", amount: 2000 });
// 7000

processTransaction(5000, { type: "withdraw", amount: 2000 });
// 3000

// insufficient balance:
processTransaction(5000, { type: "withdraw", amount: 7000 });
// 5000  (unchanged)

// insufficient balance
```

## Recommended Solving Strategy

For every problem, follow this sequence:

### Step 1 — Understand the Input

Ask: What exactly does my function receive?

```
function calculateCartTotal(products: Product[]): number
```

### Step 2 — Define the Data Types

If objects are involved, define their structure first.

```
type Product = {
    name: string;
    price: number;
};
```

### Step 3 — Define the Function

Clearly specify the parameter types and the return type.

```
function calculateCartTotal(products: Product[]): number {
    // solution
}
```

### Step 4 — Solve the JavaScript Logic

Don't overthink TypeScript. First determine the algorithm:

```
products
   -> take each price
   -> add them
   -> return total
```

### Step 5 — Use TypeScript to Make the Solution Safer

- Can this value have multiple types?
- Are there only a few valid values?
- Can a property be missing?
- What exactly should the function return?

### Step 6 — Test Edge Cases

Don't only test the provided examples. For example:

```
calculateCartTotal([]);

findProducts(products, "tablet");

getTicketPrice(5);
getTicketPrice(12);
getTicketPrice(13);
```